

Task 4: Agentic Refactoring Report

1. Agentic Tool

Tool: Antigravity

Target File: `org.apache.roller.weblogger.pojos.ObjectPermission.java`

2. Prompts Used

Prompt 1: Smell Identification

**So presently I am working on `github.com/apache/roller`
Here in `app/src/main/java/org/apache/roller/weblogger/pojos/ObjectPermission.java`
In this specific file Identifies and selects design-level smells to address
Do this properly step by step in very detail**

The above prompt is given to just identify the design smells in the system.

Prompt 2: Refactoring Planning

Refactoring Planning

- **Proposes refactoring strategies for selected smells**
 - **Briefly explains why the refactoring is appropriate**
 - **Identifies affected methods/classes due to this refactoring**
- Make sure u do refactoring on a large scale**

The above prompt is given to do refactoring using the above smells it selected itself.

Prompt 3: Refactoring Execution

Refactoring Execution

- **Produces refactored code snippets**
 - **Ensure syntactic correctness/compilability**
 - **Clearly distinguishes original vs refactored code**
- Give a read me on this - complete implementation and changes made in code**

This prompt is given to ensure that the antigravity can test the code itself and clearly explain the refactoring that happened to me.

3. Agentic Workflow Stages

3.1 Smell Identification

The agentic tool analyzed `ObjectPermission.java` and the permission hierarchy (`RollerPermission` -> `ObjectPermission`/`GlobalPermission`) and identified the following design-level smells:

Design Smell 1: Duplicated Code

- **Location:** `actions` field duplicated across `ObjectPermission` and `GlobalPermission`
- **Evidence:** Both subclasses contained identical field declarations and accessor methods
- **Impact:** Violates DRY principle; increases maintenance burden

Design Smell 2: Broken Hierarchy

- **Location:** `RollerPermission.addAction(ObjectPermission)` method
- **Evidence:** Parent class (`RollerPermission`) depends on child class (`ObjectPermission`)
- **Impact:** Creates circular dependency; violates dependency inversion principle

3.2 Refactoring Planning

The agent proposed the following refactoring strategies:

Strategy 1: Pull Up Field Refactoring

- **Target:** `actions` field and its accessors (`getActions()`, `setActions()`)
- **Rationale:** The field serves the same purpose in both `ObjectPermission` and `GlobalPermission`, indicating it belongs in the common parent `RollerPermission`
- **Affected Classes:**
 - `RollerPermission.java` (receives pulled-up members)
 - `ObjectPermission.java` (removes duplicate field/methods)
 - `GlobalPermission.java` (removes duplicate field/methods)

Strategy 2: Break Dependency Cycle

- **Target:** `addAction(ObjectPermission perm)` method signature
- **Rationale:** Parent class should not depend on specific child implementations; should use polymorphism via base class reference
- **Affected Methods:**
 - `RollerPermission.addAction()` - change parameter from `ObjectPermission` to `RollerPermission`

3.3 Refactoring Execution

The changed files are in

<https://drive.google.com/drive/folders/167fGGellR5MtNNF-3tn-1jj8wa1QymMr?usp=sharing>

Below are brief steps happened:

Change 1: RollerPermission.java

Original Code:

```
public abstract class RollerPermission extends
java.security.Permission {
    // ...
    public abstract void setActions(String actions);
    public abstract String getActions();
    // ...
    public void addActions(ObjectPermission perm) { // Dependency on
child
        List<String> newActions = perm.getActionsAsList();
        // ...
    }
}
```

Refactored Code:

```
public abstract class RollerPermission extends java.security.Permission {
    protected String actions; // Pulled up field

    public void setActions(String actions) { // Pulled up method
        this.actions = actions;
    }

    @Override
    public String getActions() { // Pulled up method
        return actions;
    }

    public void addActions(RollerPermission perm) { // Fixed
dependency
        List<String> newActions = perm.getActionsAsList();
        // ...
    }
}
```

Change 2: ObjectPermission.java

Original Code:

```

public abstract class ObjectPermission extends RollerPermission {
    protected String  actions; // Duplicated field

    @Override
    public void setActions(String actions) { // Duplicated method
        this.actions = actions;
    }

    @Override
    public String getActions() { // Duplicated method
        return actions;
    }
    // ...
}

```

Refactored Code:

```

public abstract class ObjectPermission extends RollerPermission {
    // actions field and methods removed.
    // Inherits them from RollerPermission.
}

```

Change 3: GlobalPermission.java

Original Code:

```

public class GlobalPermission extends RollerPermission {
    protected String actions; // Duplicated field

    @Override
    public void setActions(String actions) { // Duplicated method
        this.actions = actions;
    }

    @Override
    public String getActions() { // Duplicated method
        return actions;
    }
    // ...
}

```

Refactored Code:

```

public class GlobalPermission extends RollerPermission {

```

```
// actions field and methods removed.  
// Inherits them from RollerPermission.  
}
```

4. Verification Results

4.1 Compilation Check

I already asked in prompt 3 to check compilation and it checked using Automated compilation and I manually verified it.

Command: `mvn clean install`

4.2 Automated Testing

I already asked in prompt 3 to check compilation and it checked using Automated compilation and I manually verified it. My manual verification tested with `mvn test -Dtest=PermissionTest` and got Build Failure. So I used `mvn -DskipTests=false clean test` and the tests succeeded

Command: `mvn test -Dtest=PermissionTest`

Result: Build Failure by manual checking but it gave success

Command Manual: `mvn test -Dtest=PermissionTest`

Result: Success

5. Observations on Agentic Behavior

Strengths

Correctly detected hierarchy-level smells

Suggested non-trivial class-level refactoring

Maintained behavioral preservation focus

Provided syntactically valid code suggestions

Limitations

Repeated nearly identical outputs across runs

Did not deeply analyze ORM/JPA implications

No architectural-level reasoning beyond class hierarchy